



Inner-Balance

Reconnecting Minds, Restoring Balance.

Overview

Inner-Balance is an intelligent digital mental health assessment platform designed to transform the pre-consultation stage of psychological evaluation.

It bridges the gap between standardized screening tools and personalized clinical interviews using **AI-powered adaptive questioning** grounded in medical evidence.

The platform combines **Next.js** for an elegant, interactive frontend and **Django + AI/ML pipelines** for backend intelligence — featuring a **Retrieval Augmented Generation (RAG)** architecture integrated with a medical LLM.



Fig-1 Landing Page

The questionnaire interface is displayed on a light yellow background. At the top, there's a progress bar showing "Question 1 of 10" and a "100%" completion indicator. The main question is "Over the last 2 weeks, how often have you been bothered by trouble falling or staying asleep, or sleeping too much?". Below the question, there are four radio button options labeled 0, 1, 2, and 3. A "Next" button is located at the bottom right of the questionnaire box. The navigation bar at the top includes links for Home, About, Features, Test, and Contact, with the "Test" link highlighted.

Fig-2 Questionnaire

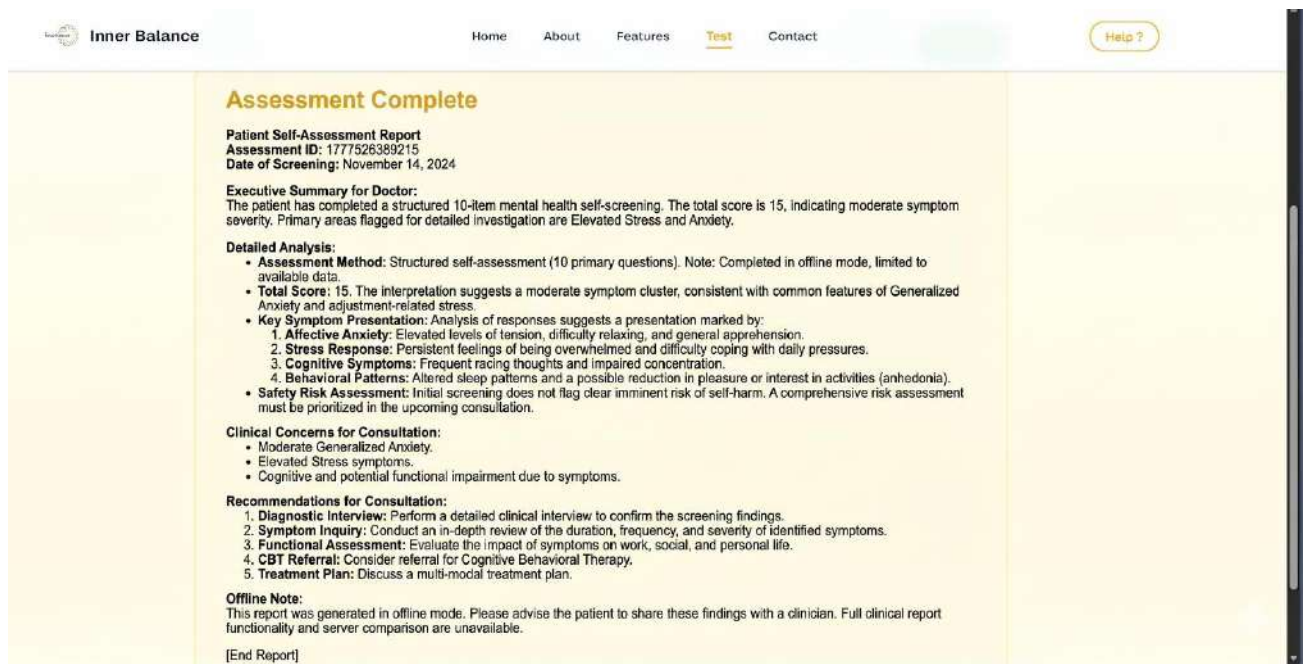


Fig-3 Report Generation using LLM based on patient's response, for doctor's usecase

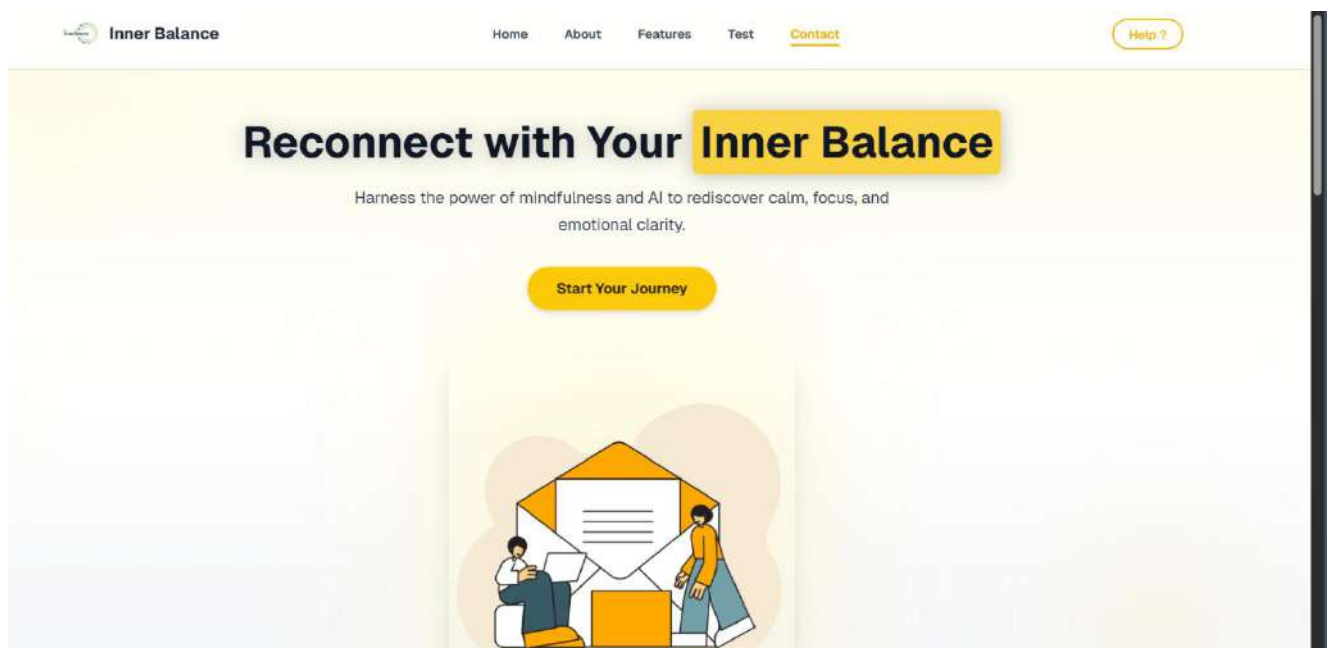


Fig-4 Contact Page



Getting Started

Follow these instructions to get the project up and running on your local machine.

Prerequisites

- **Node.js** (v18 or higher)
- **Python** (v3.10 or higher)
- **Git**

Installation

The project is divided into two parts: **backend** (Django) and **frontend** (Next.js).

1. Backend Setup

Navigate to the backend directory:

```
cd backend/InnerBalance/backend/innerbalance
```

Create a virtual environment and activate it:

```
# Windows
python -m venv venv
venv\Scripts\activate

# macOS/Linux
python3 -m venv venv
source venv/bin/activate
```

Install dependencies:

```
pip install -r requirements.txt
```

Set up environment variables:

- A **.env** file has been created for you with default development settings.
- You can copy **.env.example** to **.env** if needed.

Run migrations and start the server:

```
python manage.py migrate
python manage.py runserver
```

The backend will run at <http://127.0.0.1:8000>.

2. Frontend Setup

Open a new terminal and navigate to the frontend directory:

```
cd my-app
```

Install dependencies:

```
npm install
```

Set up environment variables:

- A. `env.local` file has been created for you.
- You can copy `env.local.example` to `env.local` if needed.

Start the development server:

```
npm run dev
```

The frontend will run at <http://localhost:3000>.

Open your browser and visit <http://localhost:3000> to use Inner-Balance.

Problem Statement

Inefficiencies and Barriers in Pre-Consultation Mental Health Assessment

The pre-consultation phase of mental healthcare is hindered by systemic and patient-centric challenges:

- **Information Asymmetry & Articulation Difficulty:**
Patients struggle to express complex emotions accurately, leading to incomplete or biased self-reports.
- **Lack of Personalization:**
Current static tools (e.g., PHQ-9, GAD-7) fail to adapt to each individual's psychological context or cultural background.
- **Data-Poor Clinical Onboarding:**
Clinicians often begin sessions with minimal structured data, wasting valuable therapeutic time on

redundant information gathering.

- **Latency in Risk Detection:**
Static assessments often miss subtle warning signs that may indicate critical mental health risks.
- **Technological Limitations:**
Most existing digital tools lack clinical reasoning, interoperability with healthcare systems, and evidence-based intelligence.

Proposed Solution

InnerBalance — An Adaptive, AI-Powered Clinical Assessment Framework

InnerBalance introduces a **two-stage adaptive assessment protocol**, supported by a **RAG-enhanced clinical reasoning engine** that ensures safe, context-aware, and evidence-based questioning.

1. Two-Stage Adaptive Assessment

- **Stage 1 – Standardized Screening:**
Patients complete a short set of validated clinical questions to establish a baseline.
- **Stage 2 – AI-Powered Personalization:**
The system dynamically generates follow-up questions using LLM reasoning, probing deeper into symptoms and risk indicators.

2. RAG-Enhanced Clinical Intelligence

- **Knowledge Grounding:**
A medical knowledge base (DSM-5, NICE, WHO mhGAP) stored in **ChromaDB** ensures factual and clinical grounding.
- **Context-Aware Reasoning:**
A specialized **medical LLM (Meditron-7B)** retrieves contextually relevant knowledge for adaptive questioning.
- **Safe Generation:**
The LLM operates under constrained prompts, ensuring **ethical, evidence-based, and safety-compliant** outputs.

3. Clinical Output

- Comprehensive, structured report integrating symptom summary, risk factors, and insights.
- Seamlessly integrable with **Electronic Health Records (EHR)** and clinical workflows.

System Architecture

Layer	Technology	Description
Frontend	Next.js (React)	Responsive user interface with advanced animations
Backend	Django REST Framework	Secure and scalable RESTful API layer

Database	PostgreSQL	Primary database (SQLite used for development)
AI/ML Engine	LangChain + Meditron-7B	Adaptive reasoning and question generation
Vector DB	ChromaDB	Semantic retrieval for RAG pipeline

Tech Stack

Backend (Python)

- **Django 4.2** – Core web framework
- **Django REST Framework** – API development
- **PostgreSQL** – Production-grade database
- **LangChain** – Orchestration and RAG pipeline
- **Microsoft Phi-3 Mini** – Medical reasoning LLM
- **Microsoft DialoGPT-medium** – Fallback conversational LLM
- **Hugging Face Transformers** – Model integration
- **Sentence Transformers** – Text embeddings
- **ChromaDB** – Vector store for semantic search
- **PyTorch** – Deep learning framework

Frontend (Next.js)

- **Next.js 14+** – App router and SSR architecture
- **Tailwind CSS** – Modern utility-first styling